Technical Changes Document: OsdagBridge Report Generator

1. Key Code and LaTeX / PyLaTeX Modifications

Specific Files Altered Under Report Generator

1. `src/osdagbridge/core/reports/report_generator.py`: Refactored to act as an orchestrator rather than a monolithic script. Stripped thousands of lines of code by delegating chapter generation to individual modules.
2. `src/osdagbridge/core/reports/chap1.py` through `chap9.py` & `executive_summary.py`: Created to segregate the document into logical chapters, improving readability and maintainability.
3. `src/osdagbridge/core/reports/styles.py`: Introduced as a centralized styling configuration module.
4. `src/osdagbridge/core/reports/charts.py`: Introduced to handle PyLaTeX and Matplotlib chart generation for data visualization.

Structure of the Centralized Formatting Configuration Module

1. **Column Width Presets**: Defines standards like `COL_LABEL` and `COL_VALUE` to ensure consistent table alignments.
2. **Document Settings**: Standardizes properties such as `PAGE_MARGIN` and `DOC_LINE_SPACING`.
3. **Table Layout & Spacing**: Encapsulates table padding (`TABLE_COL_SEP`), row height (`TABLE_EXTRA_ROW_HEIGHT`), and rule thickness (`TABLE_RULE_WIDTH`).
4. **LaTeX Longtable Headers**: Includes a standardized `lt_header` generator to manage page-breaking tables gracefully with "continued on next page" footers.

Rationale for Architectural & Styling Choices

1. **Modularity (Segregation)**: The original `report_generator.py` was overly dense, making it difficult to maintain or debug LaTeX syntax errors. Splitting it into chapter-specific files applies the Single Responsibility Principle, isolating logic and minimizing merge conflicts.
2. **Maintainability (styles.py)**: Hardcoding LaTeX parameters across multiple files causes inconsistencies. `styles.py` guarantees uniform margins, colors (`OSDAG_GREEN`), and table dimensions, making future aesthetic tweaks a one-line change.
3. **Data Visualization**: Plain tables for utilization ratios and material quantities can be hard to interpret. Generating charts dynamically via `charts.py` provides immediate visual context to the structural health and costs.
4. **Table Formatting Constraints**: The repeated 'G' characters for each row were deliberately implemented as a structural workaround to prevent page bleeding and ensure consistent line formatting and proper boundary alignment across pages.

2. Visual Enhancements & Open Polish

Several aesthetic and structural improvements were added to enhance the overall report quality:

1. **Dynamic Matplotlib Charts**:
    - Added an **Overall Utilization Ratio (UR) Bar Chart** in Section 5.5 (Design Checks), visually separating passing elements (green) from failing elements (red) against a dashed UR=1.0 limit threshold.
    - Added a **Concrete Volume vs. Reinforcement Steel Weight Chart** in Chapter 7 (Material Quantities).
2. **Improved Table Typography & Layout**:
    - Prevented tables from abruptly overflowing past the bottom margin by injecting `\needspace{5\baselineskip}` before table environments.
    - Standardized `longtable` usage across all chapters so multi-page tables have consistent, repeating headers and footers.
    - Applied uniform row stretching (`\arraystretch{1.12}`) and row padding (`\extrarowheight{0.6pt`) to give table contents breathing room, greatly improving readability over default dense LaTeX tables.
3. **Color Consistency**:
    - Standardized the use of `OSDAG_GREEN` across headers, rules, and visual elements to align with the brand guidelines.